

Protocol: A Controlled Experiment on the Impacts of Architecture Violation Hints in Code Review

1 Overview

We plan to conduct a controlled experimental design to investigate the practical impact of violation symptom hints on developers' ability to identify and refactor architectural or design issues during code review. The experiment consists of three main phases: (1) participant profiling, (2) group assignment, and (3) task execution under controlled conditions.

1.1 Phase 1: Preliminary Questionnaire (Participant Profiling)

Before the experiment, all participants were required to complete a preliminary questionnaire designed to collect background information about their software development experience. The questionnaire was administered online using Google Forms¹.

The questionnaire collected the following information:

Q1. How many years of software development experience do you have in the industry?

Q2. Please briefly describe your software development background. (Open-ended)

You may include (but are not limited to) information about your primary programming languages, size of organization, application domains, your experience with code review practices, and your familiarity with software architecture and refactoring.

The purpose of this phase was to obtain an overall distribution of participants' development experience and to support balanced group assignment in the subsequent phase.

1.2 Phase 2: Group Assignment

Based on the preliminary questionnaire responses, participants were evenly divided into two groups according to their development experience:

- **Control Group:** 15 participants
- **Treatment group:** 15 participants

The grouping ensured that both groups had comparable distributions in terms of years of development experience, professional background, and familiarity with code review practices. This grouping was conducted to minimize selection bias and reduce confounding effects caused by individual expertise differences.

1.3 Phase 3: Code Review Tasks

All participants performed a manual code review task involving the same set of three predefined code snippets. The code snippets were selected to contain known architectural or design violations (violation

¹<https://forms.gle/2jueESuAtP3QXPi19>

symptoms from our dataset) that could be addressed through refactoring. To avoid introducing additional variables, participants were explicitly instructed **NOT to use any AI-based tools or code analysis tools** during the code review process. All reviews had to be conducted manually. Each group completed the task under a different experimental condition, as described below.

1.3.1 Control Group (Questionnaire 2)

Participants in the Control Group were provided with the original code snippets without any additional hints or guidance related to violation symptoms. They were instructed to:

- Review the code manually
- Identify architecture violations
- Propose refactoring solutions based solely on their own judgment

See Questionnaire 2². No code review comments or architecture violation hints were provided.

1.3.2 Treatment group (Questionnaire 3)

Participants in the Treatment group were provided with the same code snippets with code review comments as hints describing violation symptoms. They were instructed to:

- Review the code manually
- Consider the provided violations symptom hints and identify potential violations
- Propose refactoring solutions accordingly

See Questionnaire 3³. The hints were designed to simulate tool-assisted warnings commonly encountered in real-world code review scenarios.

1.4 Data Collection

For each participant, the following data were collected:

- Written refactoring proposals for each code snippet
- Identification of relevant code locations
- Descriptions of intended structural or architectural changes

All responses were collected via online questionnaires and anonymized prior to analysis.

1.5 Post-Experiment Analysis

The refactoring solutions produced by both groups were analyzed and compared against ground-truth refactoring examples derived from expert-reviewed code review histories. These reference solutions represent validated and accepted fixes, enabling objective evaluation without reliance on subjective judgment.

The analysis aimed to assess whether the presence of violation symptom hints improves developers' ability to identify and resolve violations, thereby evaluating the practical significance of violation symptom recognition during code review.

²<https://forms.gle/qfeWXHGprGEk64NF9>

³<https://forms.gle/n9TMdj7FRTTvNE3U7>